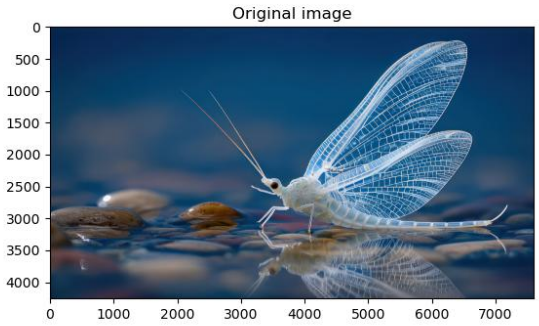
Image Transformation

A pixel-based image in RGB format can be stored as a 3D array of dimension $(h, w, 3)$ (height x width x 3), denoted as M where each (i, j) pixel in red, green, blue channel can be read from $M_{ij1}, M_{ij2}, M_{ij3}$, respectively. Think of M as a 3D array, the code `M[:,:,k]` gives you the matrix of size $h \times w$ for $k = 1, 2, 3$. The pixel intensity of an image range in 0-255 (as integers), which can also be normalized to a float number in the range [0.0,1.0] if we are going to use NumPy to handle the matrices in float type.

We can load the image using matplotlib.image, scale the pixel and intensity and plot the image as follows.



In this question, you will guided to explore three transformations on the image matrix M. For each transformation, conceptually represent $Y = T(M)$ where $T : \mathbf{R}^{h\times w\times 3} \to \mathbf{R}^{h\times w\times 3}$ and Y is the transformed image.

Tasks

1. **Reverse the order of columns in M for all three channels.** Let M_1, M_2, M_3 be the three matrices of size $(h \times w)$ obtained by slicing M on the RGB channel axis. We reverse the columns from the last to the first on each M_1, M_2, M_3. For example,

$$M_1 = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} \quad \Rightarrow \quad \begin{bmatrix} 3 & 2 & 1 \\ 3 & 2 & 1 \\ 3 & 2 & 1 \end{bmatrix}$$

   Denote $Y_1 = T_1(M)$ as the 3D array obtained by this transformation.

2. **Replace white pixels to maroon.** The white color corresponds to the color code [255,255,255] in RGB format, or scaled down to $[1, 1, 1]$. From the dragon fly image, if we would like to change the white pixels to the maroon color, what can we do? You may use a criterion to find the white pixels to check if the intensity is greater than 190/255 (some value almost 1). The maroon color code is approximately $[128, 0, 0]$ in $0 - 255$ range. Denote the result as $Y_2 = T_2(M, \texttt{threshold})$. The parameter `threshold` will be set by the user.

3. **RGB vector transform.** If you access `M[3,4,:]` you look at the $(3, 4)$ pixel and obtain an RGB vector (as a row vector in Python). Suppose that we would like to mathematically transform the RGB vector v by the following specific transformation:

$$\tilde{v} = Av = \begin{bmatrix} 0.393 & 0.769 & 0.189 \\ 0.349 & 0.686 & 0.168 \\ 0.272 & 0.534 & 0.131 \end{bmatrix}$$

In math, we would like to perform Av but to suit the Python implementation which returns v as a row vector, we can think of $\tilde{v}^T = v^T A^T$. Then, perform this transformation on every pixels in M. Denote the result as $Y_3 = T_3(M)$.

4. The entries in the RGB vector v are all less than one (as we scale M to the range $[0, 1]$). However, the transformation Av does not guarantee that $\tilde{v}$ has all entries less than one. Therefore, if we further clip the values of $\tilde{v}$ into the range $[0, 1]$, then call the new result as $Y_4 = T_4(M)$. How is T_4 related to T_3? Is T_4 linear? Note that the clipping operation applied on a real number a is that: if $a \in [0, 1]$ then return a, else if $a > 1$, return 1, else return 0. Hint: Find NumPy command for this operation.

Write a function `imagetran(M,threshold` that transforms the original image and returns Y_1, Y_2, Y_3, Y_4.

Input Format

A 3D array $M \in \mathbf{R}^{h \times w \times 3}$ where the entries are scaled from $[0, 255]$ to $[0, 1]$.

Output Format

Y_1, Y_2, Y_3, Y_4 where each $Y_k \in \mathbf{R}^{h \times w \times 3}$.

Implementation

Goal: Fill in the following function:

```
import numpy as np

def imagetran(M, whitethreshold = 190/255):
    # M is the 3D array read from an image and converted to float in range [0,1]
    # whitethreshold is a threshold in [0,1] range to approximate the white pixel
    return Y1, Y2, Y3, Y4

if __name__ == "__main__":
    import matplotlib.pyplot as plt
    import matplotlib.image as mpimg
    # your input image path
    path = 'path_to_file'

    M = mpimg.imread(path)
    print(M.shape)

    if np.issubdtype(M.dtype, np.integer):
        M = M.astype(np.float32)/255.0
        Y1, Y2, Y3, Y4 = imagetran(M)
    # ------- Insert your plotting codes here ------
    plt.title('Original image')
    plt.imshow(M)
    plt.show()
```

Hint

- Try to work with 3D array. Most NumPy commands can be broadcasted to all channels.

Warnings

- The problem is not intended to use any special packages in image processing or computer vision. It is designed to illustrate the concept of transformations in linear algebra. The tasks can be all done within NumPy package.